Appendix D. Containers as priority queues

One of the largest subset of data structures, probably the largest (by far), is the set of containers. A *container* is a collection of objects, with operations for adding, removing, and examining them. The difference between each kind of container revolve around:

1. The order in which the elements are extracted from the container.
2. Whether the elements need to be unique, or multiple instances are allowed.
3. Whether the container is associative, i.e. if it stores plain elements or associates keys with values.
4. Performance.

Points 1-3 define an abstraction of a container, in other words, its behavior - in technical jargon, its API. Once the API is fixed, implementations can still differ, though.

But for now, let’s focus on the abstract data structures. The caveat here is in the definition of priority. At a very high level, we have a black box, a *container*, that will hold our values, remove and return us a specific element every time we ask for one.

This description is generic enough that it can describe basically every type of container in data structures: it all boils down to consistently defining the priority of the